

INFORME TÉCNICO

RETO 5: SQL AVANZADO

Sistema de Gestión 'FitNet'
Javier Jiménez Alonso

1. Introducción y Justificación del Proyecto

La práctica se articula sobre la base de datos relacional «FitNet», la cual modela la actividad operativa y administrativa de un gimnasio. El esquema engloba entidades fundamentales como Socios, Membresías, Entrenadores, Clases, Equipamientos y Pagos.

Se han implementado triggers, procedimientos almacenados (con gestión de errores, transacciones y cursores) y funciones almacenadas, y un script para la generación de un dashboard.

2. Implementación de Triggers

En esta práctica se han desarrollado dos triggers fundamentales:

2.1. Trigger de Validación Preventiva (trg_validar_pago_positivo)

Tipo: BEFORE INSERT en la tabla Pago.

Finalidad: Validar las entradas de los importes al sistema.

Descripción: Este trigger intercepta cada fila que se intenta insertar en la tabla de pagos. Haciendo uso de NEW, evalúa si la cantidad del pago es inferior a cero. En tal caso, modifica la columna cantidad estableciéndola en 0. Al definirse como BEFORE se modifica antes de que el dato sea cambiado, previniendo errores.

2.2. Trigger de Automatización de Estados (trg_desactivar_socio)

Tipo: AFTER UPDATE en la tabla Membresía.

Finalidad: Sincronizar automáticamente el estado del cliente con el de su membresía.

Descripción: Al finalizar un periodo (o al detectar impago), las membresías cambian su estado a 'Caducada'. Para evitar que quede un 'Socio' activo con una membresía 'Caducada', este trigger detecta ese cambio de estado comparando los calificadores OLD.estado y NEW.estado. Se lanza un UPDATE automático sobre la tabla Socio para pasar su estado a 'Inactivo'.

3. Procedimientos Almacenados y Gestión Transaccional

Se han diseñado rutinas que encapsulan bloques de operaciones atómicas y devuelven un mensaje resolutivo haciendo un manejo avanzado de errores. Esto se apoya en el cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

3.1. Transacción con Validación: Alta de Pagos (sp_registrar_pago)

Este procedimiento recibe como parámetros de entrada los datos para ejecutar un cobro. Se verifica el identificador de membresía. Si la membresía no existe, se ejecuta un comando ROLLBACK devolviendo un mensaje de error detallado mediante un parámetro de salida OUT. Si existe, se registra el pago con INSERT y se hace COMMIT.

3.2. Operaciones Múltiples: Gestión de Equipamiento (sp_enviar_reparacion)

El mantenimiento de las máquinas de un gimnasio requiere actualizar su estado en el inventario y, a la vez, suspender su uso en cualquier clase planificada. Dentro de una única transacción se ejecutan dos acciones conjuntas: actualiza el estado de la máquina a 'Reparación' en la tabla Equipamiento y elimina los registros asociados de la tabla Clase_Equipamiento. Si cualquiera de las dos falla, el ROLLBACK deshace ambas.

4. Procesamiento Iterativo mediante Cursores

Se necesita procesar la información fila a fila. Para ello, se han definido cursores en dos procedimientos distintos:

4.1. Generador de emails para promociones (sp_emails)

Una forma de automatizar promociones por correo, se define un cursor sobre una consulta INNER JOIN que localiza los correos electrónicos de los usuarios cuyo estado asociado en la tabla Socio sea 'Activo'. El cursor recorre cada fila, concatenando los emails en una variable acumuladora.

Al finalizar el bucle, el procedimiento ejecuta un INSERT en la tabla Dashboard_Estadisticas guardando la lista de emails generada como registro de campaña. Esta operación sobre tabla es la que justifica el uso del cursor frente a una consulta directa, ya que además de leer datos, produce un efecto persistente en la base de datos.

4.2. Auditoría Interna de Activos (sp_contador_maquinas)

Se implementó un cursor que audita las máquinas operativas del complejo. Recorre los equipamientos con estado 'Operativo' y, por cada registro encontrado, ejecuta un UPDATE sobre la tabla Equipamiento para registrar la fecha de revisión, garantizando que el cursor no se limite a leer sino que produce operaciones reales sobre los datos. Al finalizar el recorrido, el parámetro OUT devuelve el recuento total de máquinas revisadas.

5. Modularización con Funciones Almacenadas

Se han implementado dos funciones que consultan datos de varias tablas para devolver un valor calculado. Ambas son reutilizadas directamente desde el procedimiento `sp_generar_dashboard`.

5.1. `fn_ingresos_totales()`

Calcula y devuelve la cantidad económica acumulada del histórico de pagos, pero únicamente de las membresías que se encuentran en estado 'Vigente'. Para ello cruza las tablas Pago y Membresia mediante INNER JOIN, filtrando por `m.estado = 'Vigente'`. Usa COALESCE para devolver 0 si no hay registros en lugar de NULL.

5.2. `fn_socios_activos()`

Cuenta la cantidad exacta de clientes con un contrato en vigor. Para ello cruza las tablas Socio y Membresia mediante INNER JOIN, aplicando una doble condición: el socio debe tener estado 'Activo' y su membresía debe tener estado 'Vigente'. Usa COUNT(DISTINCT) para evitar duplicados en caso de que un socio tenga más de una membresía registrada.

6. Script Estadístico y Dashboard

Se ha creado un script para volcar estadísticas de forma instantánea de la situación del gimnasio.

Previamente se definió la tabla `Dashboard_Estadisticas` con los campos `id_informe`, `fecha_generacion`, `total_socios_activos`, `ingresos_totales` y `mensaje_estado`. Seguidamente, se implementó el procedimiento `sp_generar_dashboard()`. Se llama a las funciones (ingresos y activos), se almacena sus resultados en variables, y se inserta una nueva fila con el timestamp (`NOW()`). Al acabar, con un SELECT se muestran todos los datos juntos; es como un log para no tener que buscar uno a uno todos los campos que pienso que son necesarios.

Campo	Tipo	Descripción
<code>id_informe</code>	INT AUTO_INCREMENT	Clave primaria del registro
<code>fecha_generacion</code>	DATETIME	Momento de la ejecución (<code>NOW()</code>)
<code>total_socios_activos</code>	INT	Resultado de <code>fn_socios_activos()</code>
<code>ingresos_totales</code>	DECIMAL(10,2)	Resultado de <code>fn_ingresos_totales()</code>
<code>mensaje_estado</code>	VARCHAR(255)	Usado por <code>sp_emails</code> para registrar campañas

7. Conclusión

Se ha logrado tener un mínimo de automatización e independencia en el gimnasio. Se ha tenido en cuenta controles de flujo y se han creado variables locales, excepciones y transacciones para que el control de la base de datos sea más estable ante fallos y errores.

Los triggers garantizan la integridad de los datos de forma transparente sin necesidad de modificar el código de aplicación. Los procedimientos con transacción aseguran que operaciones que afectan a varias tablas se ejecuten de forma atómica. Los cursores permiten procesar y persistir información fila a fila. Las funciones modularizadas simplifican el script del dashboard, que actúa como un log centralizado del estado del sistema.